16 貪婪演算法 解答

16-E01 活動選擇與交換

推導

最早結束是 (1,3)，選它。下一個相容最早結束為 (4,8)，再選 (7,9)？(4,8) 與 (7,9) 重疊，不可同時；因此解為 (1,3)、(4,8)，共 2。先選 (2,5) 後可選 (7,9)，也是 2，但不超過貪婪解。

結論與檢核

交換論證不是說每個最佳解都必須選第一個，而是可把某個最佳解首項換成最早結束且不減大小。

16-E02 硬幣貪婪反例

推導

Greedy 先取 5+5，剩 2，再取 1+1，共 4 枚。最佳為 4+4+4，共 3 枚。

結論與檢核

反例證明只按最大面額的 greedy 不正確。若要建立 greedy，需針對面額族證明 canonical property，不能從常見 1、5、10、25 外推。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16-E03 Huffman 合併

推導

先合 C1+D1=2，再合 B2+2=4，再合 A4+4=8。可取 A=0、B=10、C=110、D=111。

結論與檢核

加權長度=4・1+2・2+1・3+1・3=14。C、D 互換或同頻率分支左右互換可得到不同最優碼，總成本相同。

16-E04 加權活動選擇何時需要 DP

推導

活動按結束時間排序為 short,long,tail。對每個 j，令 p(j) 是結束時間不晚於活動 j 開始時間的最後一個活動索引，D[j] 為前 j 個活動的最大價值：採 1-based 活動索引，無相容前驅時 p(j)=0，D[0]=0，D[j]=max(D[j-1], value_j+D[p(j)])。

16 貪婪演算法 解答 （續）

結論與檢核

short 的 p=0：D1=max(0,1)=1。long 的 p=0：D2=max(1,100)=100。tail 的 p=2：D3=max(100,100+1)=101。最優集合 {long,tail}。最早結束 greedy 先選 short，之後可選 tail，只得到 2。高價值活動可能較晚結束，因此「最大數量」的交換證明不能直接控制「總價值」。加權版本有最佳子結構，但局部 finish-time 選擇不再安全；必須比較包含與排除當前活動。

16-E05 增廣性不是遺傳性的推論

推導

任取 I 中集合，其所有子集仍列在 I；例如 {b,c} 的 {b}、{c}、∅ 都獨立，故具遺傳性。取 A={a}、B={b,c}，|A|=1<2，但 A∪{b}={a,b} 與 A∪{c}={a,c} 都不在 I，故增廣性失敗。

結論與檢核

greedy 依權重選 a，之後 b、c 均不能加入，得到 {a}，價值 10；{b,c} 本身獨立且價值 18，所以 greedy 勝不過最優。若系統真是 matroid，greedy-choice 會保證某最優集含 a；本例正是少了增廣性。證明 greedy 正確時，不能只檢查空集獨立或子集封閉。還須驗證增廣性，並把候選加入後的 independence oracle 成本算入總時間。

16-E06 截止容量與完整排程

推導

依 penalty 順序 r,s,t,u,v。接受 r 時前綴(1,1,1)；接受 s 後(1,2,2)；試 t 變(2,3,3)，N1>1，拒絕，恢復(1,2,2)；接受 u 後(1,2,3)；試 v 變(1,3,4)，N2>2，拒絕。準時集{r,s,u}，排在槽 1,2,3，t,v 放4,5，都逾期。最小罰金10+7=17。

結論與檢核

一般證明令 k=max{t:N_t(A)=t}，0包含其中。若 |A|<|B|，A 中 deadline>k 的工作有 |A|−k 個，B 中至少 |B|−k 個，所以存在 x∈B\A 且 deadline>k。對 t≤k 加入 x 不改 N_t；對 t>k，N_t(A)≤t−1，故加入至多1仍可行。遺傳性與空集條件亦成立，故是 matroid，正 penalty 的降冪 greedy 全域最優。k 不是 N_t(A)=N_t(B) 的最後相等點，兩個前綴差可以跨零而不取零。

16 貪婪演算法 解答 （續）

16-E07 Huffman 交換的符號與單字元邊界

推導

交換前這兩葉貢獻1・1+5・3=16，交換後1・3+5・1=8，新減舊為−8。一般為 fx・da+fa・dx−fx・dx−fa・da =(fa−fx)(dx−da)≤0，因 fa≥fx、dx≤da。若寫成(fa−fx)(da−dx)則是舊減新，不能把它標成新減舊。

結論與檢核

只有一個葉時沒有合併，樹深度0使樹論成本0；若實作約定字元碼為0，出現 fx 次需 fx 位元，必須另外記錄訊息長度以解碼。合併權重總和等於加權葉深度的定理仍適用樹深度模型，但不含單字元實作強制一位元的額外約定。兩個成本模型不可混用。

原創驗算程式附錄: greedy_revision.py

```
"""Exact traces and exchange checks for the Chapter 16 revision pages."""

from dataclasses import dataclass
from heapq import heapify, heappop, heappush
from itertools import combinations, product
from functools import lru_cache


@dataclass(frozen=True)
class Activity:
    name: str
    start: int
    finish: int
    value: int = 1


def activity_selection(activities):
    ordered = sorted(activities, key=lambda activity: (activity.finish, activity.start))
    if not ordered:
        return [], []
    chosen = [ordered[0]]
    rejected = []
    for activity in ordered[1:]:
        if activity.start >= chosen[-1].finish:
            chosen.append(activity)
        else:
            rejected.append(activity)
    return chosen, rejected


def weighted_activity_dp(activities):
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16 貪婪演算法 解答 （續）

```
    ordered = sorted(activities, key=lambda activity: activity.finish)
    end_times = [activity.finish for activity in ordered]
    previous = []
    for current, activity in enumerate(ordered):
        compatible = [
            index
            for index, earlier in enumerate(ordered[:current])
            if earlier.finish <= activity.start
        ]
        previous.append(max(compatible) if compatible else -1)

    best = [0] * (len(ordered) + 1)
    choice = [False] * (len(ordered) + 1)
    for count in range(1, len(ordered) + 1):
        include = ordered[count - 1].value + best[previous[count - 1] + 1]
        exclude = best[count - 1]
        if include > exclude:
            best[count] = include
            choice[count] = True
        else:
            best[count] = exclude

    selected = []
    count = len(ordered)
    while count:
        if choice[count]:
            selected.append(ordered[count - 1])
            count = previous[count - 1] + 1
        else:
            count -= 1
    return best[-1], tuple(reversed(selected)), tuple(end_times)
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16 貪婪演算法 解答 （續）

```
def huffman(frequencies):
    heap = [(weight, serial, name) for serial, (name, weight) in
enumerate(frequencies.items())]
    heapify(heap)
    merges = []
    serial = len(heap)
    while len(heap) > 1:
        left_weight, _, left_tree = heappop(heap)
        right_weight, _, right_tree = heappop(heap)
        weight = left_weight + right_weight
        merges.append((left_tree, right_tree, weight))
        heappush(heap, (weight, serial, (left_tree, right_tree)))
        serial += 1
    tree = heap[0][2]

    codes = {}

    def collect(node, prefix):
        if isinstance(node, str):
            codes[node] = prefix or "0"
            return
        collect(node[0], prefix + "0")
        collect(node[1], prefix + "1")

    collect(tree, "")
    cost = sum(frequencies[name] * len(code) for name, code in codes.items())
    return tree, merges, codes, cost


def independent_tasks(tasks):
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

16 貪婪演算法 解答 （續）

```
    return all(
        deadline >= position
        for position, deadline in enumerate(sorted(task[1] for task in tasks), 1)
    )


def task_schedule(tasks):
    selected = []
    rejected = []
    for task in sorted(tasks, key=lambda item: (-item[2], item[1], item[0])):
        if independent_tasks(selected + [task]):
            selected.append(task)
        else:
            rejected.append(task)
    early_order = sorted(selected, key=lambda item: (item[1], item[0]))
    late_penalty = sum(item[2] for item in rejected)
    return selected, rejected, early_order, late_penalty


def hereditary_greedy_counterexample():
    elements = {"a": 10, "b": 9, "c": 9}
    independent = {
        frozenset(),
        frozenset({"a"}),
        frozenset({"b"}),
        frozenset({"c"}),
        frozenset({"b", "c"}),
    }
    assert all(
        subset in independent
        for item in independent
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16 貪婪演算法 解答 (續)

```
        for size in range(len(item) + 1)
        for subset in map(frozenset, combinations(item, size))
    )
    greedy = []
    for item, _ in sorted(elements.items(), key=lambda pair: -pair[1]):
        if frozenset(greedy + [item]) in independent:
            greedy.append(item)
    assert greedy == ["a"]
    assert elements["b"] + elements["c"] == 18 > elements["a"]
    return greedy, (elements["b"] + elements["c"])


@lru_cache(None)
def exhaustive_merge_cost(weights):
    if len(weights) < 2:
        return 0
    return min(
        weights[i]+weights[j]+exhaustive_merge_cost(tuple(sorted(
            [weight for k, weight in enumerate(weights) if k not in (i,j)]
            + [weights[i]+weights[j]]
        )))
        for i in range(len(weights)) for j in range(i+1,len(weights))
    )


def audit_exchange_and_schedules():
    assert activity_selection(()) == ([], [])
    old_cost, new_cost = 1*1+5*3, 1*3+5*1
    delta = (5-1)*(1-3)
    assert new_cost-old_cost == delta == -8
    assert new_cost-old_cost != (5-1)*(3-1)
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16 貪婪演算法 解答 （續）

```
    merge_cases = 0
    for n in range(2, 6):
        for weights in product(range(1,4),repeat=n):
            _, merges, _, cost = huffman(dict(zip("ABCDE",weights)))
            assert cost == sum(merge[2] for merge in merges)
            assert cost == exhaustive_merge_cost(tuple(sorted(weights)))
            merge_cases += 1

    augmentation_cases = 0
    for n in range(1,5):
        for deadlines in product(range(1,n+1),repeat=n):
            tasks = tuple((str(i), d, n-i) for i,d in enumerate(deadlines))
            feasible = [set(ids) for k in range(n+1)
                        for ids in combinations(range(n),k)
                        if independent_tasks([tasks[i] for i in ids])]
            selected, _, _, _ = task_schedule(tasks)
            optimum=max(sum(tasks[i][2] for i in ids) for ids in feasible)
            assert sum(task[2] for task in selected)==optimum
            for a in feasible:
                for b in feasible:
                    if len(a)>=len(b):
                        continue
                    tight=max(t for t in range(n+1)
                              if sum(tasks[i][1]<=t for i in a)==t)
                    options=[i for i in b-a if tasks[i][1]>tight]
                    assert options
                    assert all(independent_tasks([tasks[i] for i in a|{x}]) for x in options)
                    augmentation_cases += 1
    print(f"PASS negative_exchange_delta={delta} exhaustive_huffman={merge_cases}
deadline_augmentation_pairs={augmentation_cases}")
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

16 貪婪演算法 解答 （續）

```
def main():
    activities = (
        Activity("a", 1, 3),
        Activity("b", 2, 5),
        Activity("c", 3, 6),
        Activity("d", 5, 7),
        Activity("e", 7, 9),
        Activity("f", 8, 10),
    )
    chosen, rejected = activity_selection(activities)
    assert [item.name for item in chosen] == ["a", "c", "e"]
    assert [item.name for item in rejected] == ["b", "d", "f"]

    weighted = (
        Activity("short", 0, 2, 1),
        Activity("long", 1, 3, 100),
        Activity("tail", 3, 4, 1),
    )
    earliest_finish = [weighted[0], weighted[2]]
    best_value, best_set, _ = weighted_activity_dp(weighted)
    assert sum(item.value for item in earliest_finish) == 2
    assert best_value == 101 and [item.name for item in best_set] == ["long", "tail"]

    frequencies = {"A": 5, "B": 2, "C": 1, "D": 3}
    tree, merges, codes, huffman_cost = huffman(frequencies)
    assert [merge[2] for merge in merges] == [3, 6, 11]
    assert huffman_cost == 20

    tasks = (
        ("a", 3, 9),
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16 貪婪演算法 解答 （續）

```
        ("b", 1, 8),
        ("c", 3, 7),
        ("d", 2, 6),
        ("e", 1, 5),
    )
    selected, rejected_tasks, early_order, late_penalty = task_schedule(tasks)
    assert [item[0] for item in selected] == ["a", "b", "c"]
    assert [item[0] for item in rejected_tasks] == ["d", "e"]
    assert [item[0] for item in early_order] == ["b", "a", "c"]
    assert late_penalty == 11

    hereditary_greedy, hereditary_best = hereditary_greedy_counterexample()
    assert hereditary_greedy == ["a"] and hereditary_best == 18
    audit_exchange_and_schedules()

    print(
        "PASS "
        f"activity={[item.name for item in chosen]} "
        f"weighted_best={best_value}:{[item.name for item in best_set]} "
        f"huffman_merges={[merge[2] for merge in merges]} "
        f"huffman_codes={codes} cost={huffman_cost} "
        f"task_early={[item[0] for item in early_order]} "
        f"penalty={late_penalty} "
        f"hereditary_greedy={hereditary_greedy}/{hereditary_best}"
    )


if __name__ == "__main__":
    main()
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study